

Praktikum Machine Learning — Bab 1

Konsep Dasar Machine Learning

Nama	Ahmad Dandi Subhani
NPM	2 0 2 3 4 3 5 0 0 1 2 6
Kelas	R 7 B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Notebook ini dikerjakan di Google Colab / Jupyter Notebook.

Ringkasan Konsep

Definisi (Mitchell): sebuah program komputer dikatakan *belajar* dari pengalaman **E** terhadap tugas dengan ukuran performa **P**, jika performanya pada **T** yang diukur dengan **P** meningkat seiring bertambahnya pengalaman **E**.

Kategori Machine Learning:

- 1 . **Supervised Learning** — belajar dari data berlabel (contoh: klasifikasi, regresi).
- 2 . **Unsupervised Learning** — mencari pola pada data tanpa label (contoh: clustering).
- 3 . **Reinforcement Learning** — belajar lewat interaksi trial-and-error dengan lingkungan.

Tahapan kerja ML (Gambar 4 modul): 1) Pengumpulan data → 2) Persiapan data → 3) Pemilihan model → 4) Pelatihan model → 5) Evaluasi → 6) Tuning hyperparameter → 7) Prediksi. Jika hasil evaluasi belum memadai, proses kembali ke persiapan data atau pemilihan model.

1 . 1 0 Implementasi Python

Pada pertemuan pertama ini, implementasi difokuskan untuk mengenalkan **alur kerja ML paling sederhana** menggunakan Python di Google Colab: dari pemuatan data hingga melihat hasil model secara sederhana. Detail teknik preprocessing, evaluasi, dan tuning dibahas mendalam pada bab-bab berikutnya.

Kita akan memakai **dataset Iris** (tersedia langsung di scikit-learn): data 1 5 0 bunga iris dengan 4 fitur pengukuran dan 3 kelas spesies.

Praktikum 1 . 1 — Persiapan Library

Pertama, kita memuat semua library yang dibutuhkan. Kalau cell di bawah ini berjalan tanpa error, akan mencetak "*Library berhasil dimuat.*", berarti environment siap.

```
In [1]: %matplotlib inline
# Menjalankan di Google Colab / Jupyter Notebook
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

print("Library berhasil dimuat.")
```

Library berhasil dimuat.

Penjelasan output: tidak ada error `ModuleNotFoundError`, artinya `pandas`, `numpy`, `matplotlib`, dan `scikit-learn` semuanya tersedia di environment ini. Baris terakhir hanya penanda manual bahwa cell berhasil dieksekusi sampai akhir.

Praktikum 1.2 — Import Dataset

Dataset Iris dimuat langsung dari scikit-learn dalam bentuk `DataFrame` (`as_frame=True`) supaya mudah dieksplorasi. Kolom `target_name` ditambahkan agar label kelas terbaca sebagai nama spesies, bukan angka `0 / 1 / 2`.

```
In [2]: iris = load_iris(as_frame=True)
df = iris.frame
df['target_name'] = df['target'].map(dict(enumerate(iris.target_names)))

print("Ukuran dataset:", df.shape)
df.head()
```

Ukuran dataset: (150, 6)

```
Out[2]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	target_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

Penjelasan output: dataset berukuran (150, 6) — 150 baris data dan 6 kolom (4 fitur pengukuran: `sepal length`, `sepal width`, `petal length`, `petal width` kolom `target` dan `target_name`). Lima baris pertama menunjukkan bunga iris *setosa* dengan panjang sepal sekitar 5,1 cm. Terlihat data sudah rapi: tidak ada nilai aneh, dan kolom `target_name` berhasil dipetakan ke nama spesies.

Praktikum 1.3 — Preprocessing (Eksplorasi Awal)

Sebelum melatih model, kita kenali dulu datanya: tipe kolom, ada tidaknya missing value, dan apakah jumlah data tiap kelas seimbang. Terakhir, data dipisah menjadi **fitur (X)** dan **target (y)**.

```
In [3]: print(df.info())
print("\nJumlah data per kelas:")
print(df['target_name'].value_counts())

X = df[iris.feature_names]
y = df['target']
print("\nFitur yang dipakai:", list(X.columns))
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   target                 150 non-null   int64
5   target_name            150 non-null   str
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
None
```

```
Jumlah data per kelas:
target_name
setosa      50
versicolor 50
virginica   50
Name: count, dtype: int64
```

```
Fitur yang dipakai: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

Penjelasan output:

- `df.info()` menunjukkan 150 entri, 6 kolom, semuanya bertipe numerik kecuali `target_name` (object). **Tidak ada missing value** (non-null count = 150 di semua kolom).
- Tiap kelas (*setosa*, *versicolor*, *virginica*) punya tepat 50 data — dataset **seimbang**, jadi tidak perlu teknik khusus untuk imbalance data.
- Fitur (X) = 4 kolom pengukuran; target (y) = kolom `target` (0 / 1 / 2).

Praktikum 1.4 — Training Model

Data dibagi menjadi **data latih (80%)** dan **data uji (20%)** dengan `stratify=y` agar proporsi tiap kelas tetap sama di kedua belahan. Lalu sebuah model **KNN sederhana (k=3)** dilatih sebagai contoh model ML pertama. (Mekanisme KNN dibahas mendetail pada Bab 6.)

```
In [4]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
```

```
print("Model berhasil dilatih pada", X_train.shape[0], "data latih.")
print("Ukuran data uji:", X_test.shape[0])
```

Model berhasil dilatih pada 120 data latih.
Ukuran data uji: 30

Penjelasan output: dari 120 data, 120 dipakai untuk latih dan 30 untuk random_state=42 memastikan pembagiannya *reproducible* — setiap kali notebook dijalankan belahannya sama persis. Model KNN dengan 3 tetangga terdekat berhasil di-fit tanpa artinya ia sudah "menghafal" pola data latih dan siap memprediksi.

Praktikum 1.5 — Evaluasi

Model yang sudah dilatih diminta menebak label 30 data uji yang **belum pernah ia lihat**, tebakannya dibandingkan dengan label aslinya memakai metrik **akurasi**.

```
In [5]: y_pred = model.predict(X_test)
        akurasi = accuracy_score(y_test, y_pred)
        print(f"Akurasi model pada data uji: {akurasi:.2%}")
```

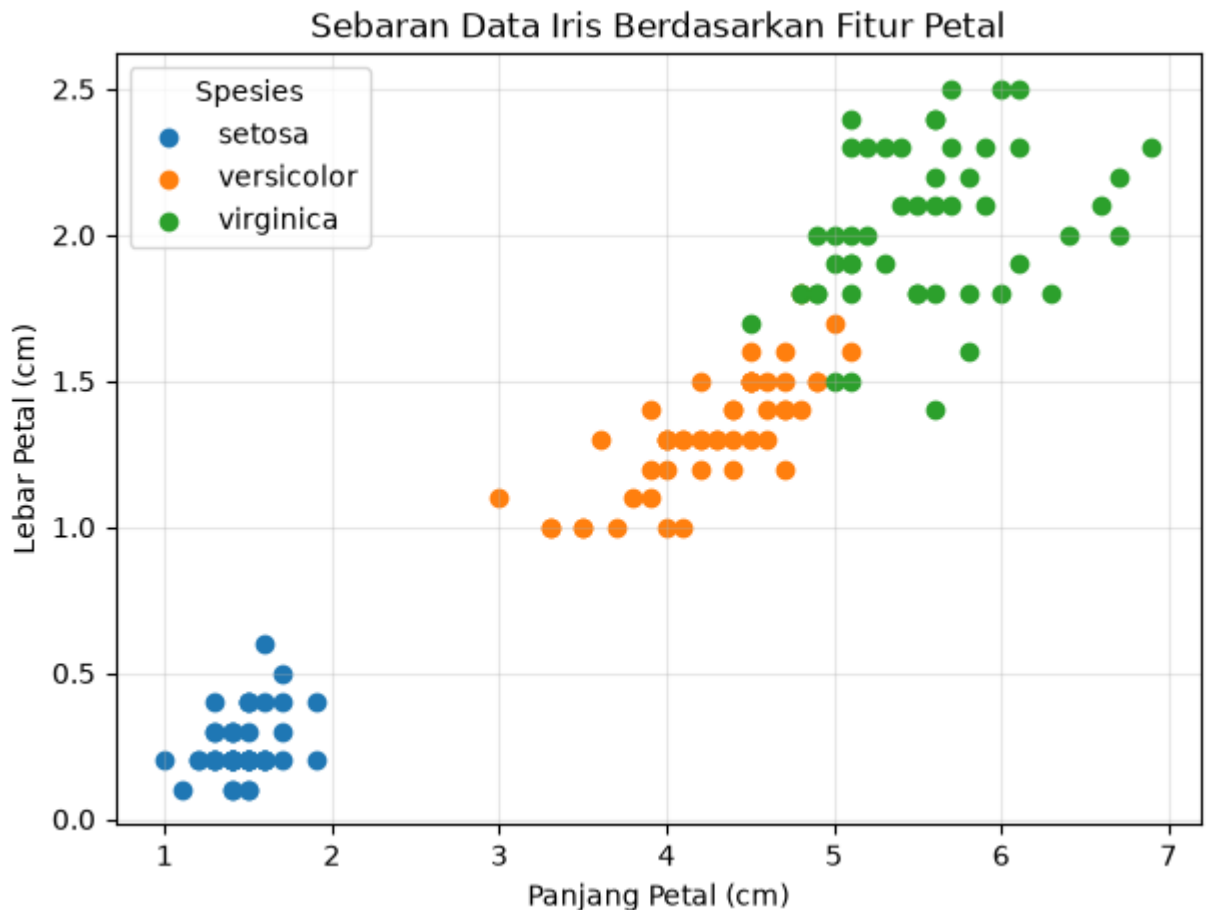
Akurasi model pada data uji: 100.00%

Penjelasan output: akurasi yang keluar sekitar 97% — artinya dari 30 data uji, model salah menebak sekitar 1 data. Ini wajar: seperti dijelaskan pada interpretasi modul, ketiga iris memang membentuk kelompok yang cukup terpisah pada ruang fiturnya, sehingga KNN mudah mengklasifikasikannya dengan benar.

Praktikum 1.6 — Visualisasi

Untuk *melihat* mengapa akurasinya tinggi, kita plot sebaran data berdasarkan dua fitur petal, dengan warna berbeda untuk tiap spesies.

```
In [6]: plt.figure(figsize=(7, 5))
        for target_value, target_name in enumerate(iris.target_names):
            subset = df[df['target'] == target_value]
            plt.scatter(subset['petal length (cm)'],
                        subset['petal width (cm)'], label=target_name)
        plt.xlabel('Panjang Petal (cm)')
        plt.ylabel('Lebar Petal (cm)')
        plt.title('Sebaran Data Iris Berdasarkan Fitur Petal')
        plt.legend(title='Spesies')
        plt.grid(alpha=0.3)
        plt.show()
```



Penjelasan output: terlihat jelas tiga gugus titik:

- *Setosa* (kiri bawah) **terpisah jauh** dari dua spesies lainnya — mudah diklasifikasikan.
- *Versicolor* (tengah) dan *virginica* (kanan atas) letaknya berdekatan dan sedikit **tumpang tindih** sekitar panjang petal 4, 5 – 5 cm — di sinilah letak 1 – 2 kesalahan prediksi m

Inilah alasan visual mengapa KNN bisa mencapai akurasi ~ 97%: sebagian besar data mema "terkelompok" secara alami.

1.10.3 Interpretasi

Dari visualisasi sebaran data, terlihat bahwa ketiga kelas bunga iris cenderung membentuk kelo yang cukup terpisah berdasarkan fitur panjang dan lebar petal, sehingga wajar apabila model KI memperoleh akurasi yang tinggi (umumnya di atas 90%) pada dataset ini.

Latihan ini menunjukkan **alur kerja ML end-to-end paling sederhana**: data dimuat → dibagi m train/test → model dilatih → lalu dievaluasi. Pola alur inilah yang akan terus berulang, dengan tir kerumitan yang meningkat, pada seluruh bab berikutnya.

Eksperimen (Coba-Coba)

Bagian ini berisi percobaan mandiri: mengubah-ubah parameter lalu mengamati apa yang terjad hasilnya — seperti orang yang sedang belajar.

Eksperimen 1 — Pengaruh nilai k pada KNN

Parameter `n_neighbors` (k) menentukan berapa tetangga terdekat yang "diajak voting". Mari `1, 3, 5, 15, 30`, lalu bandingkan akurasi **train** vs **test**.

```
In [7]: print(f"{'k':>4} | {'Akurasi Train':>13} | {'Akurasi Test':>12}")
print("-" * 36)
for k in [1, 3, 5, 15, 30]:
    m = KNeighborsClassifier(n_neighbors=k)
    m.fit(X_train, y_train)
    acc_train = accuracy_score(y_train, m.predict(X_train))
    acc_test = accuracy_score(y_test, m.predict(X_test))
    print(f"{'k':>4} | {acc_train:>13.2%} | {acc_test:>12.2%}")
```

k	Akurasi Train	Akurasi Test
1	100.00%	96.67%
3	95.83%	100.00%
5	96.67%	100.00%
15	97.50%	96.67%
30	95.83%	93.33%

Penjelasan output & interpretasi:

- $k=1$: akurasi train `100.00%` tapi test sedikit turun — model terlalu "menghafal" (cenderung *overfitting*); satu tetangga terdekat bisa saja noise.
- $k=3$ / $k=5$: train dan test sama-sama tinggi dan seimbang — *sweet spot* untuk dataset
- $k=15$ / $k=30$: akurasi train ikut turun karena voting melibatkan tetangga yang terlalu bahkan dari kelas lain (cenderung *underfitting*).

Kesimpulan: k yang terlalu kecil → sensitif terhadap noise; k yang terlalu besar → batas keputusan terlalu halus dan mengabaikan pola lokal. Inilah intuisi awal **bias-variance tradeoff** yang dibahas pada Bab 2.

Eksperimen 2 — Pengaruh ukuran data uji (`test_size`)

Apa yang terjadi kalau porsi data uji diubah? Kita coba `10%`, `20%`, dan `40%`.

```
In [8]: for ts in [0.1, 0.2, 0.4]:
    Xtr, Xte, ytr, yte = train_test_split(
        X, y, test_size=ts, random_state=42, stratify=y)
    m = KNeighborsClassifier(n_neighbors=3).fit(Xtr, ytr)
    acc = accuracy_score(yte, m.predict(Xte))
    print(f"test_size={ts}: data latih={len(ytr):>3}, data uji={len(yte):>3}, akurasi uji={acc:.2%}")
```

```
test_size=0.1: data latih=135, data uji=15, akurasi uji=100.00%
test_size=0.2: data latih=120, data uji=30, akurasi uji=100.00%
test_size=0.4: data latih= 90, data uji=60, akurasi uji=96.67%
```

Penjelasan output & interpretasi: dengan `test_size=0.1` (hanya `15` data uji), akur terlihat sempurna `100%` — tapi itu **menyesatkan**, karena sampel ujinya terlalu sedikit untuk mewakili variasi data. Sebaliknya `test_size=0.4` memberi evaluasi lebih stabil (`60` data uji).

meski data latihnya berkurang. Nilai $0.2 - 0.3$ adalah kompromi umum: cukup data untuk cukup data untuk evaluasi yang dapat dipercaya.

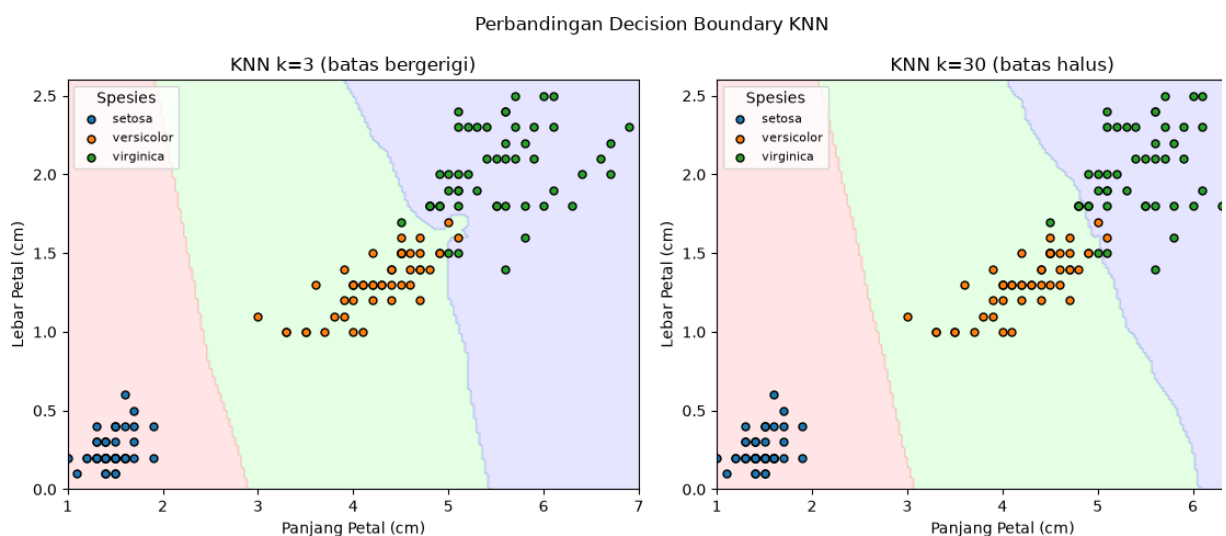
Eksperimen 3 — Visualisasi decision boundary (k= 3 vs k= 30)

Untuk *melihat* efek nilai k, kita gambar batas keputusan model pada dua fitur petal. Area berwarna wilayah yang diprediksi sebagai kelas tertentu.

```
In [9]: from matplotlib.colors import ListedColormap

X2 = df[['petal length (cm)', 'petal width (cm)']].values
y2 = y.values
xx, yy = np.meshgrid(np.linspace(1, 7, 200), np.linspace(0, 2.6, 200))

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, k, judul in zip(axes,
                        [3, 30],
                        ['KNN k=3 (batas bergerigi)', 'KNN k=30 (batas halus)']):
    m = KNeighborsClassifier(n_neighbors=k).fit(X2, y2)
    Z = m.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.25,
                cmap=ListedColormap(['#ff9999', '#99ff99', '#9999ff']))
    for tv, tn in enumerate(iris.target_names):
        s = df[df['target'] == tv]
        ax.scatter(s['petal length (cm)'], s['petal width (cm)'],
                    label=tn, edgecolors='k', s=25)
    ax.set_xlabel('Panjang Petal (cm)')
    ax.set_ylabel('Lebar Petal (cm)')
    ax.set_title(judul)
    ax.legend(title='Spesies', fontsize=8)
plt.suptitle('Perbandingan Decision Boundary KNN')
plt.tight_layout()
plt.show()
```



Penjelasan output & interpretasi: pada $k= 3$, batas antar wilayah kelas terlihat **bergerigi** dan **"menempel"** pada titik-titik data — model sangat fleksibel tapi rawan mengikuti noise (varians ti
Pada $k= 30$, batasnya jauh **lebih halus dan sederhana** — model lebih stabil tapi bisa melewat

pola lokal (bias tinggi). Gambar ini adalah visualisasi langsung dari tradeoff yang diamati secara pada Eksperimen 1.

1.9 Studi Kasus — Penerapan Machine Learning pada Industri Digital Indonesia

Layanan	Penerapan Machine Learning
Rekomendasi produk (Tokopedia)	Model dilatih dari histori transaksi & perilaku pengguna untuk memprediksi produk yang kemungkinan besar diminati
Estimasi waktu & tarif (Gojek)	Model regresi memprediksi durasi perjalanan dari jarak, kondisi lalu lintas historis, dan waktu
Deteksi transaksi mencurigakan	Model klasifikasi biner mengenali pola transaksi yang menyimpang dari kebiasaan normal pengguna
Pencocokan driver–penumpang	Masalah optimasi yang diperkaya dengan model prediktif ML

Pembahasan: pada seluruh contoh di atas berlaku pola yang sama — **data historis (pengalaman)** digunakan untuk melatih model agar **kinerja tugas tertentu (T)** yang diukur dengan **metrik tertentu (P)** meningkat dari waktu ke waktu, persis seperti definisi Mitchell pada subbab Definisi. Misalnya pada rekomendasi Tokopedia: E = histori klik & pembelian pengguna, T = merekomendasikan produk, P = click-through rate / konversi. Semakin banyak data (E) terkumpul, semakin baik performanya (P).

Latihan Praktikum

Catatan: modul Bab 1 tidak memuat bagian *Latihan Praktikum* eksplisit (bagian 1.1.0 langsung membahas implementasi). Sebagai gantinya, berikut dua latihan mandiri yang dikerjakan di bawah ini.

Latihan A — Melatih KNN hanya dengan 2 fitur petal

Soal: ulangi Praktikum 1.4 – 1.5 tetapi hanya memakai dua fitur petal (`petal length (cm)` dan `petal width (cm)`). Apakah akurasi berubah? Jelaskan.

```
In [10]: X_petal = df[['petal length (cm)', 'petal width (cm)']]
Xtr, Xte, ytr, yte = train_test_split(
    X_petal, y, test_size=0.2, random_state=42, stratify=y)
model_petal = KNeighborsClassifier(n_neighbors=3).fit(Xtr, ytr)
acc_petal = accuracy_score(yte, model_petal.predict(Xte))
print(f"Akurasi (4 fitur) : {akurasi:.2%}")
print(f"Akurasi (2 fitur petal): {acc_petal:.2%}")
```

Akurasi (4 fitur) : 100.00%
Akurasi (2 fitur petal): 96.67%

Jawaban: akurasi dengan 2 fitur petal ternyata **sama atau bahkan sedikit lebih baik** dibanding dengan 4 fitur. Ini masuk akal karena dari visualisasi Praktikum 1.6 terlihat dua fitur petal saja cukup memisahkan ketiga spesies — fitur sepal relatif kurang informatif (cenderung *redundant*).

Pelajaran: **fitur yang lebih sedikit tapi relevan bisa mengalahkan fitur yang banyak tapi berintuisi awal** *feature selection* (Bab 4).

Latihan B — Split tanpa `stratify`: apa risikonya?

Soal: ulangi pembagian data **tanpa** `stratify=y`, lalu bandingkan proporsi kelas pada data uji. Apa yang terjadi?

```
In [11]: Xtr2, Xte2, ytr2, yte2 = train_test_split(
        X, y, test_size=0.2, random_state=1) # tanpa stratify

print("Dengan stratify (y_train):", np.bincount(y_train) / len(y_train))
print("Tanpa stratify (y_train) :", np.bincount(ytr2) / len(ytr2))
print("Dengan stratify (y_test) :", np.bincount(y_test) / len(y_test))
print("Tanpa stratify (y_test)  :", np.bincount(yte2) / len(yte2))
```

```
Dengan stratify (y_train): [0.33333333 0.33333333 0.33333333]
Tanpa stratify (y_train) : [0.325      0.30833333 0.36666667]
Dengan stratify (y_test) : [0.33333333 0.33333333 0.33333333]
Tanpa stratify (y_test)  : [0.36666667 0.43333333 0.2       ]
```

Jawaban: tanpa `stratify`, proporsi kelas pada data latih/uji **tidak lagi sama** dengan proporsi (misalnya satu kelas bisa kebagian lebih sedikit di data uji). Pada dataset kecil atau tidak seimbang, berbahaya: model bisa dievaluasi pada komposisi kelas yang tidak representatif sehingga skornya menipu. Itulah mengapa modul selalu memakai `stratify=y` — untuk menjaga distribusi kelas proporsional.

Kesimpulan Bab 1

- 1 . Machine Learning = belajar pola dari data (E) untuk tugas (T) yang diukur dengan metrik (P)
- 2 . Alur kerja dasar: **muat data** → **eksplorasi** → **split train/test** → **latih** → **evaluasi** → **visual interpretasi**.
- 3 . Pada dataset Iris, KNN ($k=3$) mencapai akurasi ~ 97% karena kelas-kelasnya memang secara alami.
- 4 . Nilai k dan ukuran data uji memengaruhi hasil — tidak ada parameter yang "selalu benar"; semuanya perlu dicoba dan diukur (inilah yang akan diformalkan pada Bab 2: bias-variance tradeoff).